Chapter 0a: SQL Review and ACID Transactions

Structured Query Language (SQL) lets us describe the data we want and the changes we want to make. A query specifies a result; the database system decides how to produce it. This is a quick review of the operations we will use throughout the course. (1)

For a longer introduction with interactive examples, work through the W3Schools SQL Tutorial. Start with SELECT, WHERE, INSERT INTO, UPDATE, and DELETE, then continue with joins and grouping. The tutorial covers several database products, so some details differ from SQLite. (2)

Tables, Rows, and Keys

A **table** stores rows with named columns. A **primary key** identifies a row. A **foreign key** connects a row to a row in another table.

Here is a small SQLite example. Run the statements in order in a new database. SQLite's foreign-key setting applies to the current connection.

```
PRAGMA foreign_keys = ON;

CREATE TABLE owner (
    id INTEGER PRIMARY KEY,
    name TEXT NOT NULL
);

CREATE TABLE pet (
    id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    age INTEGER NOT NULL CHECK (age >= 0),
    owner_id INTEGER NOT NULL REFERENCES owner(id)
);

INSERT INTO owner (id, name)
VALUES (1, 'Alex'), (2, 'Blair');

INSERT INTO pet (id, name, age, owner_id)
VALUES (1, 'Casey', 3, 1), (2, 'Milo', 7, 2);
```

The schema gives the database rules to enforce. A pet needs a name, a non-negative age, and an owner who exists. The foreign key stores an identifier rather than another copy of the owner's name.

Read the Data

```
SELECT name, age
FROM pet
WHERE age >= 4
ORDER BY age DESC, name;
```

This returns Milo, age seven. `WHERE` filters rows. `ORDER BY` specifies their order; without it, the result has no promised display order.

A **join** combines matching rows from related tables:

```
SELECT pet.name AS pet_name, owner.name AS owner_name
FROM pet
JOIN owner ON pet.owner_id = owner.id
ORDER BY pet.id;
```

pet_name	owner_name
Casey	Alex
Milo	Blair

Grouping lets us summarize several rows. This query counts pets for each owner who has at least one pet:

```
SELECT owner.name, COUNT(*) AS pet_count
FROM owner
JOIN pet ON pet.owner_id = owner.id
GROUP BY owner.id, owner.name
ORDER BY owner.name;
```

`COUNT(*)` counts rows in each group. An owner with no pets does not appear in this inner join.

Change the Data

```
UPDATE pet
SET age = 4, owner_id = 2
WHERE id = 1;
```

Casey is now four and belongs to Blair. The row keeps its original primary key. An update changes the selected fields; it does not require replacing the whole record.

```
DELETE FROM pet WHERE id = 2;
```

This removes Milo. Both `UPDATE` and `DELETE` use `WHERE` to limit the affected rows. Without that condition, the operation applies to every row in the table.

Transactions and ACID

A **transaction** groups database work into a unit that can be committed or rolled back. **Commit** makes the transaction's changes permanent. **Rollback** cancels its uncommitted changes.

ACID names four transaction properties: Atomicity, Consistency, Isolation, and Durability. They describe what we need from a database when operations fail or several programs use it at once. (3)

Property	Meaning	Why it matters
Atomicity	All of a transaction's changes happen, or none do.	A failed transfer must not debit one account without crediting the other.
Consistency	A transaction preserves the database's defined rules.	A saved pet must not refer to an owner who does not exist.
Isolation	Concurrent transactions have controlled visibility and interference.	Two users changing the same records need predictable results.
Durability	A successful commit survives a subsequent failure.	A confirmed save must not disappear when the program restarts.

The major SQL database systems implement ACID transactions, including SQLite and PostgreSQL (often called Postgres). Support does not mean every configuration provides the strongest possible guarantees. Transaction boundaries, isolation levels, and storage settings still matter. SQLite provides ACID transactions even though the database lives in a file; it does not need a separate database server to do so. (4)(5)

Consistency depends on rules we actually specify. The database can reject a negative age because we declared a `CHECK` constraint. It cannot infer every rule of our application. Correct transaction code and declared constraints work together.

Isolation is especially important when two requests overlap. Different isolation levels permit different interactions; the strongest, serializable isolation makes completed transactions behave as though they ran one at a time. A transaction is not automatically a request for that strongest level. (3)(5)

Try a Rollback

After the earlier statements, Casey belongs to Blair. Try changing the owner inside a transaction:

```
BEGIN;
UPDATE pet SET owner_id = 1 WHERE id = 1;
SELECT name, owner_id FROM pet WHERE id = 1;
ROLLBACK;
SELECT name, owner_id FROM pet WHERE id = 1;
```

The first query shows owner 1. The second shows owner 2: the rollback cancelled the reassignment. Repeat the sequence with `COMMIT` instead of `ROLLBACK` to keep the change. PostgreSQL uses the same basic transaction commands. (5)

An application groups related changes between those boundaries. On success it commits. On failure it rolls back. A failure in one statement does not mean every database automatically cancels every earlier statement in the transaction; the application's error-handling path must finish the transaction correctly.

Practice

1. Add a third owner and a pet belonging to that owner. Use a join to display their names together.
2. Query the pets in descending age order. Include a second ordering field to make ties predictable.
3. Attempt to assign Casey to owner 999. Explain which declared rule rejects the change.
4. Update Casey's age inside a transaction, inspect it, and roll it back. Verify the saved age afterward.

Considerations

- Why does a foreign key use the owner's identifier rather than the owner's name?
- Would atomicity alone prevent an application from committing the wrong account balances?
- What could go wrong if two users both read an available seat before either saves a reservation?

References

1. Wikipedia contributors. "SQL." https://en.wikipedia.org/wiki/SQL
2. W3Schools. "SQL Tutorial." https://www.w3schools.com/sql/
3. Wikipedia contributors. "ACID." https://en.wikipedia.org/wiki/ACID
4. SQLite. "SQLite Is Transactional." https://www.sqlite.org/transactional.html
5. PostgreSQL Global Development Group. "Transactions." PostgreSQL documentation. https://www.postgresql.org/docs/current/tutorial-transactions.html